Allow static data members in local and unnamed classes

Document #:	P3588R1
Date:	2025-05-17
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Brian Bi <bbi10@bloomberg.net>

Contents

§ 1 Abstract

Local and unnamed classes (as well as classes nested within) are not permitted to declare static data members. This restriction dates back to C++98, when there was no way to provide a definition for such a member. In modern C++, static data members can have inline definitions, so this rationale is obsolete. Static data members can be useful in local classes for the same reasons why they are useful in non-local classes, so this paper proposes to allow them in C++29.

§ 2 Revision history

§ 2.1 R1

— Updated ship vehicle to C++29

— Expanded discussion of initialization order, providing three possible designs

— Expanded discussion of implementation experience, including discussion of bug in implementation

— Added discussion of ordering involving complete-class contexts

— Updated wording to account for the fact that static variable declarations would be able to overlap under this proposal

§ 3 Background

C++03 §[class.static.data] stated as follows:

> […] Unnamed classes and classes contained directly or indirectly within unnamed classes shall not contain `static` data members. [*Note*: this is because there is no mechanism to provide the definitions for such `static` data members.]
>
> […] A local class shall not have `static` data members.

No rationale is explicitly provided for the latter restriction. However, we can see that "there is no mechanism to provide the definitions" applies to static data members of local classes as well. There was one additional possible issue, which is the need to generate an external symbol based on the identity of the local class, but it seems that this applies equally well to functions (*i.e.*, it seems that a member function of a local class in an inline function would need to have a weak external symbol since its address could be taken), and in any case, thanks to [N2657], unnamed and local classes became valid template arguments from C++11 and it is certain that all implementations can generate appropriate mangled names for local classes and members of local and unnamed classes. There do not appear to be any remaining technical obstacles to allowing such classes to have static data members.

[CWG728] proposed to relax some restrictions on local classes; more specifically, to allow them to be templates, declare member templates, declare friends, and declare static data members. It was determined that this was a feature request, and a paper was submitted, [P2044R2]. However, that paper addressed only the issue of member templates. *This* paper deals only with the simpler feature of static data members.

§ 4 Why this is useful

C++ entities should generally be declared in the narrowest scope in which they are needed, which may be a single function. For example, in Google Test, the `TEST` macro is used to define a single function which then verifies some expected properties. If a class is required only for that single test, it is convenient to declare it as a local class. It can be desirable for such a local class to have a static data member, such as a static data member that tracks the number of live objects of the class.

Here is another example: suppose that a C++ library defines a custom version of the tuple protocol in which a tuple-like class is expected to declare a static data member named `tuple_size`, initialized to a compile-time constant. If a unit test needs to declare such a class and pass it as a template argument to the component under test (which is expecting it to be a tuple), it would be convenient to be able to scope that class to the smallest enclosing block.

In the former case, the static data member would be declared `static inline`. In the latter case, it would be declared `static constexpr`, which makes it implicitly inline. A non-inline static data member in a local class cannot be defined, but I see no reason not to also allow it. It could be used just for its type (*i.e.*, in unevaluated contexts only). Allowing such a variable to inhabit class scope would make it more narrowly scoped than if it were a local `extern` variable inhabiting the enclosing function.

Note that although the current language does not permit static data members in local classes, it does permit static local variables in member functions of local classes. That is, the Meyer singleton pattern can be contained within an outer block scope. This workaround for the inability to declare static data members in local classes adds verbosity in requiring a function to be defined and called. Meyer singletons also trade off some performance for safety: additional instructions are executed every time the function containing the static local variable is called (unless the variable is constant-initialized), but most cases of initialization order bugs are prevented. Although the additional safety is useful, having to incur changes in performance as a cost of moving static variables within a block scope violates the C++ design principle that *you don't pay for what you don't use*.

§ 5 Proposal

§ 5.1 Summary

Allow static data members to be declared by local and unnamed classes without restriction, except unnamed classes that have a typedef name for linkage purposes. That is, the following would remain ill formed:

```
typedef struct {
    inline static int x = 0;
} S;
```

§9.2.4 [dcl.typedef][1]p5 places various restrictions on such classes in the vein of requiring them to be "C-like": for example, they can't have member functions nor member typedefs.

Since static data members don't exist in C, I don't propose to drop this restriction.

Likewise, restrictions on anonymous unions (§11.5.2 [class.union.anon]p1) are left untouched by this proposal. Being a compatibility feature with C, they aren't allowed to declare any members other than public non-static data members.

Note that static data members may be thread-local, and therefore thread-local data members will also become allowed in local classes under this proposal. However, the current Standard doesn't specify initialization order for thread-local namespace-scope variables ([CWG2914], [CWG2928]). The intent of this proposal is to be orthogonal to those Core issues: just as static data members of local classes are to be initialized in the same time as if they were at namespace scope, the same is meant to hold for thread-local members of local classes.

§ 5.2 Initialization order

I propose that static data members of local classes follow the same initialization order rules as if their classes were non-local: for example, if `x`, `y`, and `z` are three inline variables and in every translation unit they appear in that order, and `x` and `z` belong to non-local classes while `y` belongs to a local class, then `y` is initialized after `x` and before `z`, unless `y`'s enclosing function is templated, in which case `y` could be initialized in any order relative to `x` and `z`. I believe that this rule is intuitive and any other rule would be a source of bugs: if a later-initialized static variable could be visible at the point of declaration of an earlier-initialized one, the earlier-initialized variable could use the later-initialized variable's value before the latter has been initialized.

It is worth noting that lexical initialization order conflicts with a principle of C++ classes, that their behavior should not change depending on the location of the class's complete-class contexts. Consider the following example:

```
struct C {
    int f();

     static inline int a = f();   // `b` is declared later and not yet
        initialized
};

int C::f() {
    struct S {
        static inline int b = 1;
    };
    return S::b;
}
```

Here, `a` would be initialized prior to `b`; `f` would return either 0 or 1 (see §6.9.3.2 [basic.start.static]p3).

Evidently, lexical initialization order can't prevent all initialization order bugs, but the alternative, namely to guarantee that static data members of class `C` are initialized before

static data members appearing within complete-class contexts of `C`, would result in a similar bug when `f` is defined in line:

```
struct C {
    void f() {
        struct S {
            static inline int b = 1;
        };
    }

     static inline int a = f();   // `b` is declared in `f` and not yet
        initialized
};
```

Given that neither initialization order rule is perfect, I still advocate lexical order as the easiest to understand. A third alternative that avoids initialization order issues completely, but reduces the usefulness of the feature, is to require static data members of local classes to be constant-initialized and have constant destruction.[2]

To summarize, the three options are

1. Non-templated non-local static variables (including data members of local classes) are initialized in lexical declaration order;
2. Non-templated non-local static variables are initialized in declaration order, considering declarations appearing in complete-class contexts of any class to follow those appearing in non-complete-class-contexts of that class;
3. All static data members of local classes must be constant-initialized and have constant destruction.

§ 5.3 What about functions that are never used?

If a function with internal or no linkage is never called and never has its address taken, implementations can currently skip code generation for the function. However, since I propose that the initialization of a static data member of a local class should occur as described in §6.9.3 [basic.start] (*i.e.* upon program or thread startup, unless deferred) it would be surprising if such an initialization having side effects could be skipped if the enclosing function isn't called. Therefore, if there are any static data members defined inside the function, code generation must be done for those static data members and potentially for lambdas reachable from them.[3]

Note that for a local class, the definition of each member is always be instantiated eagerly when the enclosing function is instantiated; see Note 3 to §13.9.2 [temp.inst]p2. I don't propose any carve-out to this rule for static data members.

§ 6 Implementation experience

GCC supports static data members in unnamed and local classes as an extension when `-fpermissive` is used. GCC does not allow such members to be explicitly declared `inline`, but this seems to be an oversight: the error message says "'inline' specifier invalid for variable 'x' declared at block scope". GCC does allow the member to be `constexpr`, and generates an appropriate definition in that case (*i.e.*, the address can be taken).

I implemented this feature partially, including dynamic initialization of inline static data members, as a patch to Clang 20 (May 4, 2025). This implementation passes the Clang unit tests but has the following known bugs:

1. The initialization order is not correct; complete-class contexts are parsed later than the rest of the enclosing class, so the order in which static data member initializers are emitted corresponds to option 2 above, rather than option 1.

2. Static data members of local classes are currently not emitted in cases such as

   ```
   // x appears at namespace scope
   int x = [] { struct S { static inline int y = 0; }; return S::y; }();
   ```

   because the code that iterates over definitions to emit them misses closure types that were injected into the enclosing namespace scope. (As a result, a program containing the above translation unit fails to link.)

In order to ensure lexical initialization order, it seems that an implementation might need to store the location of each non-templated static data member found while parsing a top-level class definition (*i.e.*, one that does not have any enclosing class scope) and then sort them at the end of the class definition. This would be novel but not extremely difficult to implement. Such lists (of which multiple might be produced for every namespace-scope declaration) could also be used to fix the second bug.

§ 7 Proposed wording

Wording is relative to [N5008].

Modify §6.9.3.3 [basic.start.dynamic]p2 as follows to account for the fact that a definition of a static data member of a local class can be nested within the definition of some other variable requiring dynamic initialization.

> For two definitions `D` and `E` of non-block variables with static storage duration, ~~A declaration~~ `D` is *appearance-ordered* before ~~a declaration~~ `E` if
>
> — `D` appears in the same translation unit as `E`, or
>
> — the translation unit containing `E` has an interface dependency on the translation unit containing `D`,
>
> and ~~in either case prior to~~the *init-declarator* of `D` ends before the *init-declarator* of `E`.

Strike §11.4.9.3 [class.static.data]p2. (The first sentence is redundant with §9.2.2 [dcl.stc]p8.)

> ~~A static data member shall not be `mutable` ([dcl.stc]). A static data member shall not be a direct member ([class.mem]) of an unnamed ([class.pre]) or local ([class.local]) class or of a (possibly indirectly) nested class ([class.nest]) thereof.~~

Strike §11.6 [class.local]p4.

> ~~[*Note 2*: A local class cannot have static data members ([class.static.data]). — *end note*]~~

§ 8 Acknowledgements

Bengt Gustafsson, Arthur O'Dwyer, and Corentin Jabot provided valuable feedback on the motivation, wording, and implementation for this paper.

§ 9 References

[CWG2914] Brian Bi. 2024-06-20. Unclear order of initialization of static and thread-local variables.
https://wg21.link/cwg2914

[CWG2928] CWG. 2024-08-16. No ordering for initializing thread-local variables.
https://wg21.link/cwg2928

[CWG728] Faisal Vali. 2008-10-05. Restrictions on local classes.
https://wg21.link/cwg728

[N2657] John Spicer. 2008-06-10. Local and Unnamed Types as Template Arguments.
https://wg21.link/n2657

[N5008] Thomas Köppe. 2025-03-15. Working Draft, Programming Languages — C++.
https://wg21.link/n5008

[P2044R2] Robert Leahy. 2020-04-14. Member Templates for Local Classes.
https://wg21.link/p2044r2

---

1. All citations to the Standard are to working draft N5008 unless otherwise specified.↩

2. All `constexpr` static variable declarations would necessarily satisfy this criterion, but there are others that would as well, such as the example of an instance counter, defined as `static inline int count = 0;`; the destruction of `count` itself is trivial.↩

3. If the behavior of my (partial) implementation exhibits any deviation from these rules, the deviation is unintentional.↩